



UNIVERSITY OF TARTU

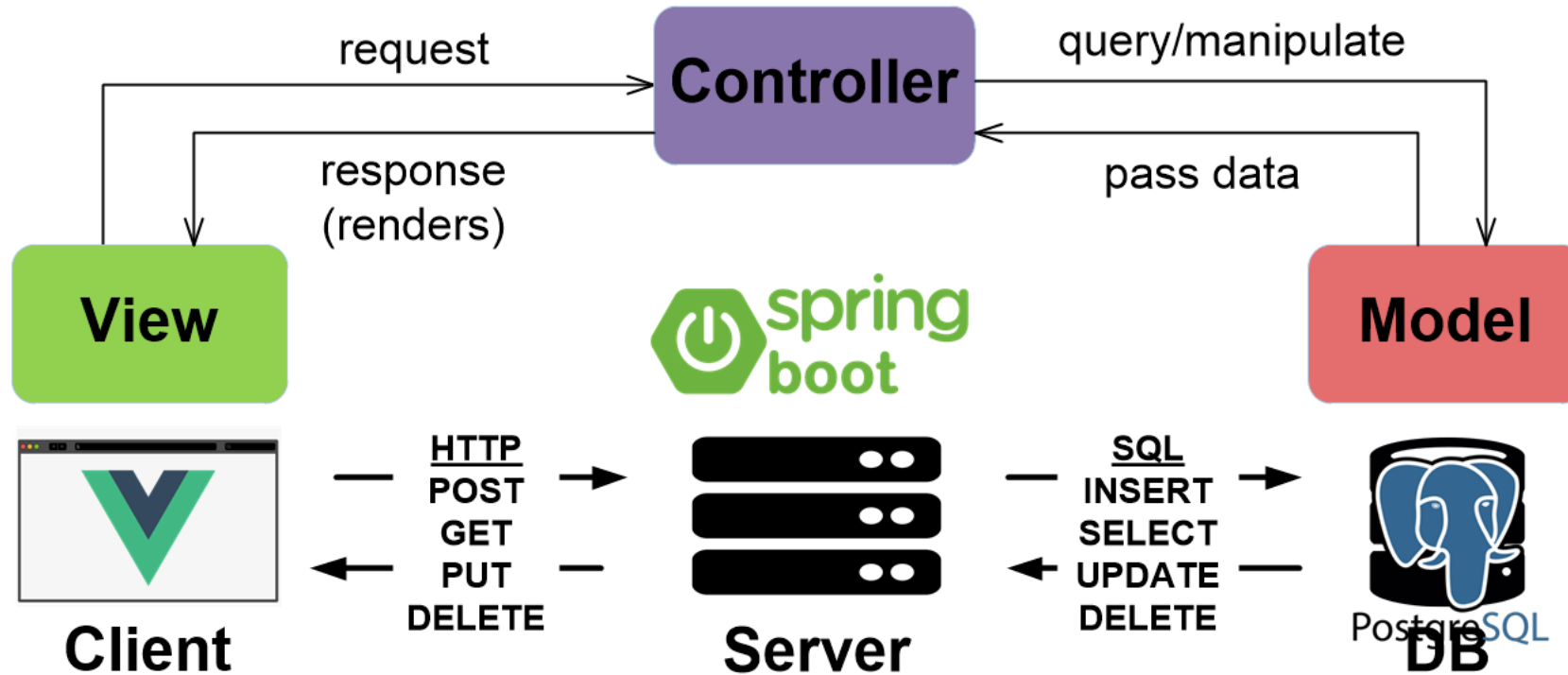


Enterprise System Integration (MTAT.03.229)

LECTURE 3: RESTFUL API – CRUD OPERATIONS - II

**MOHAMAD GHARIB
UNIVERSITY OF TARTU**

Building an MVC application - recap



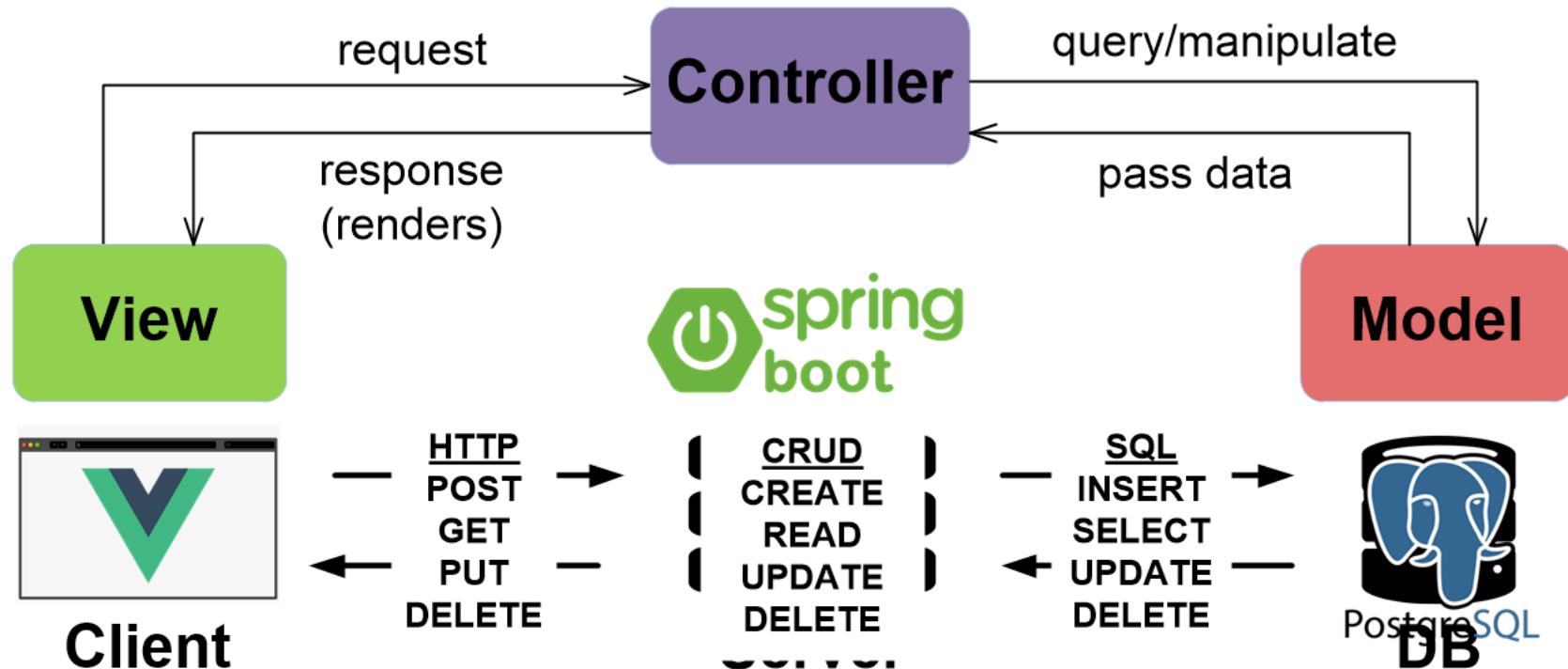
The View - a front-end (Vue.js) that provides the user interface.

The Controller - a backend (Spring Boot) allows for handling front-end requests.

The Model - a database (Postgres) that contain the data of our App.

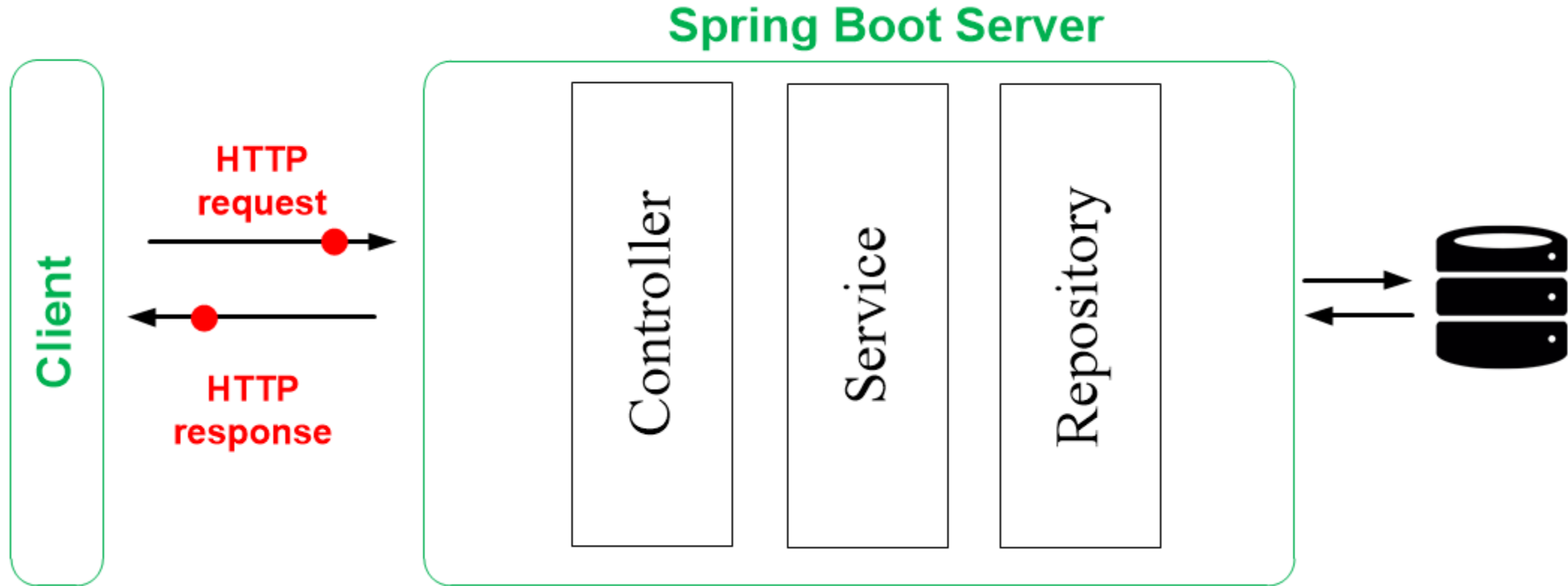
CRUD operations - recap

CRUD
Create
Read
Uppdate
Deleate



CRUD operations are the **four basic** operations of persistent storage.

REST Architecture – CSR pattern - recap



Controller-Service-Repository pattern

The CSR pattern - practical example

In this example, we have two entities:

- Product
- Order

esi.~.products

Product (Entity)

ProductController

ProductService

ProductRepository



The CSR pattern - practical example

In this example, we have two entities:

- Product
- Order

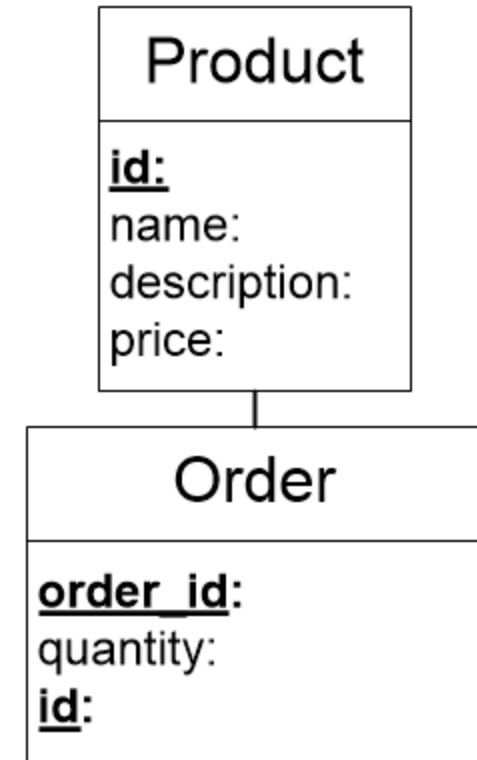
esi.~.orders

Order (Entity)

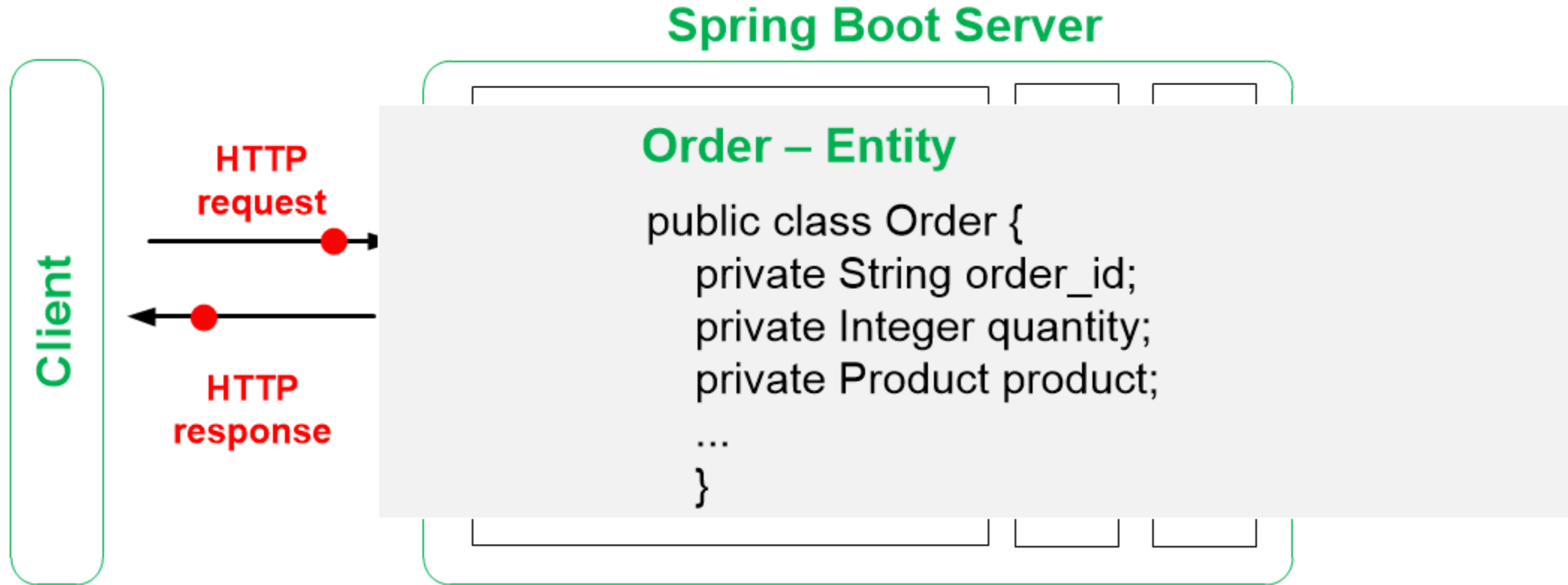
OrderController

OrderService

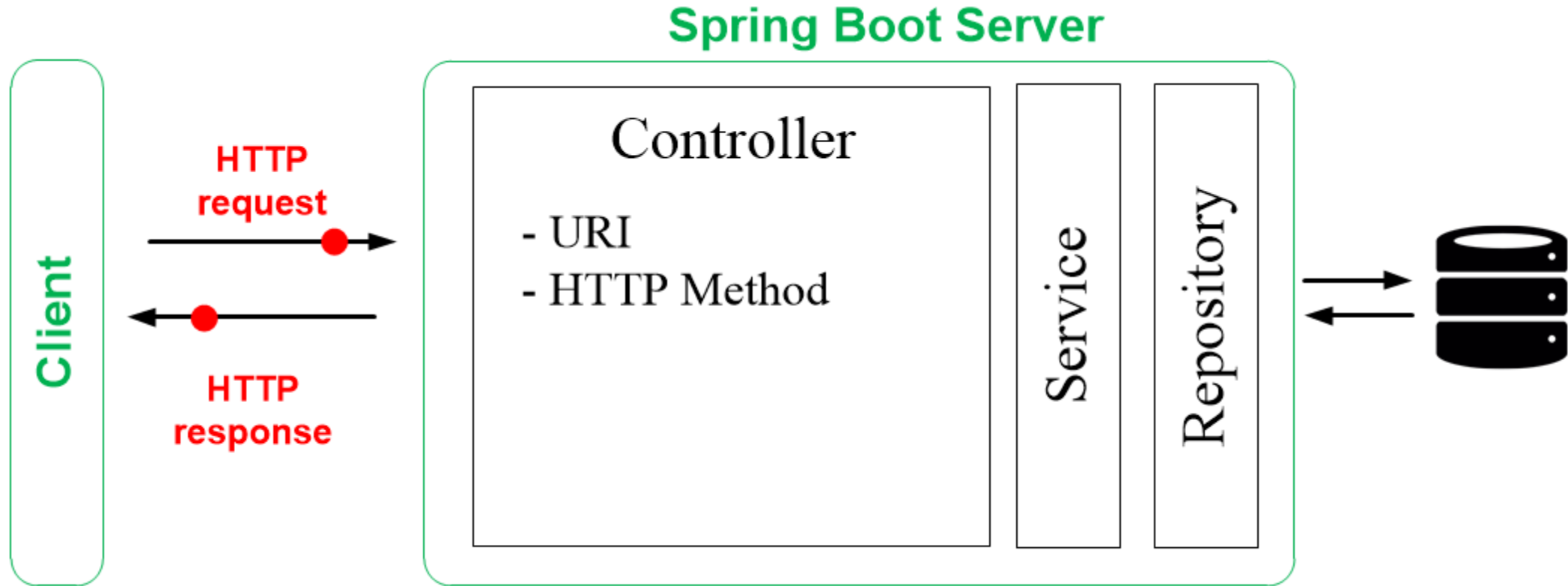
OrderRepository



REST Architecture – Order Entity

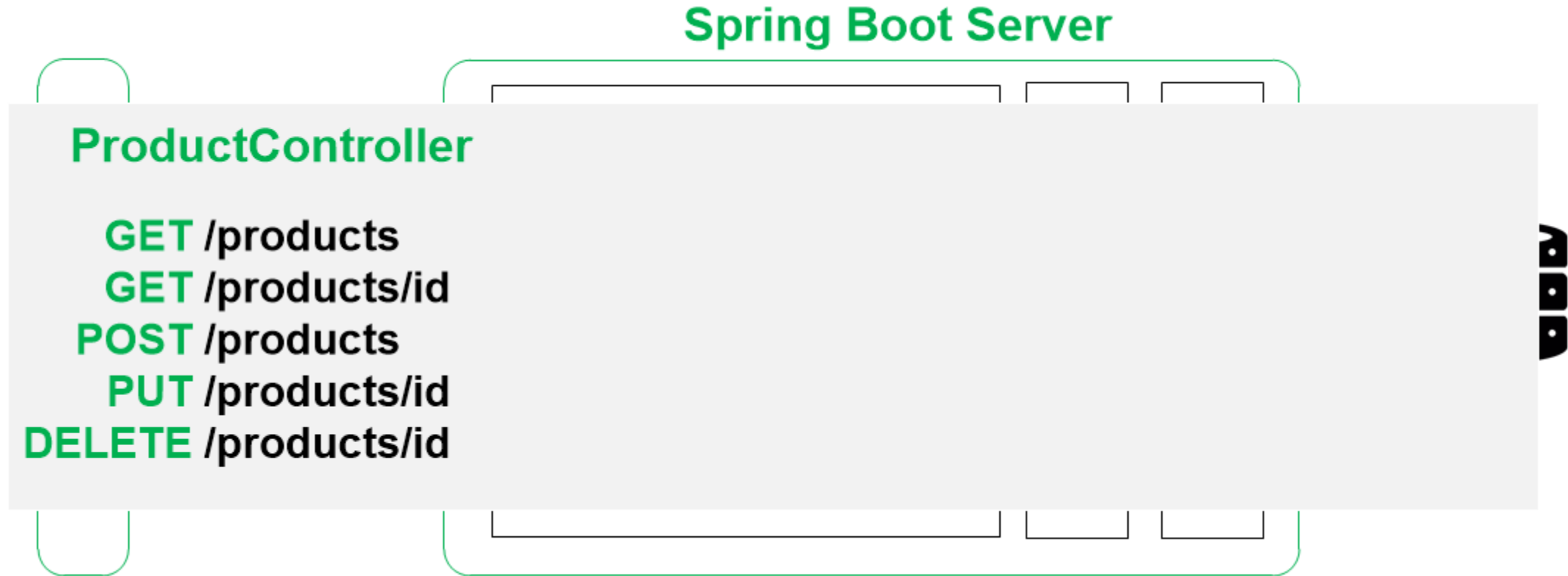


REST Architecture - Controller



A **Controller** maps **clients requests** to corresponding **services/functionalities** (Java methods).

REST Architecture - Controller



A Controller maps clients requests to corresponding services/functionalities (Java methods).

REST Architecture - Controller

Spring Boot Server

OrderController

GET /products/**id**/orders

GET /products/id/orders/**oid**

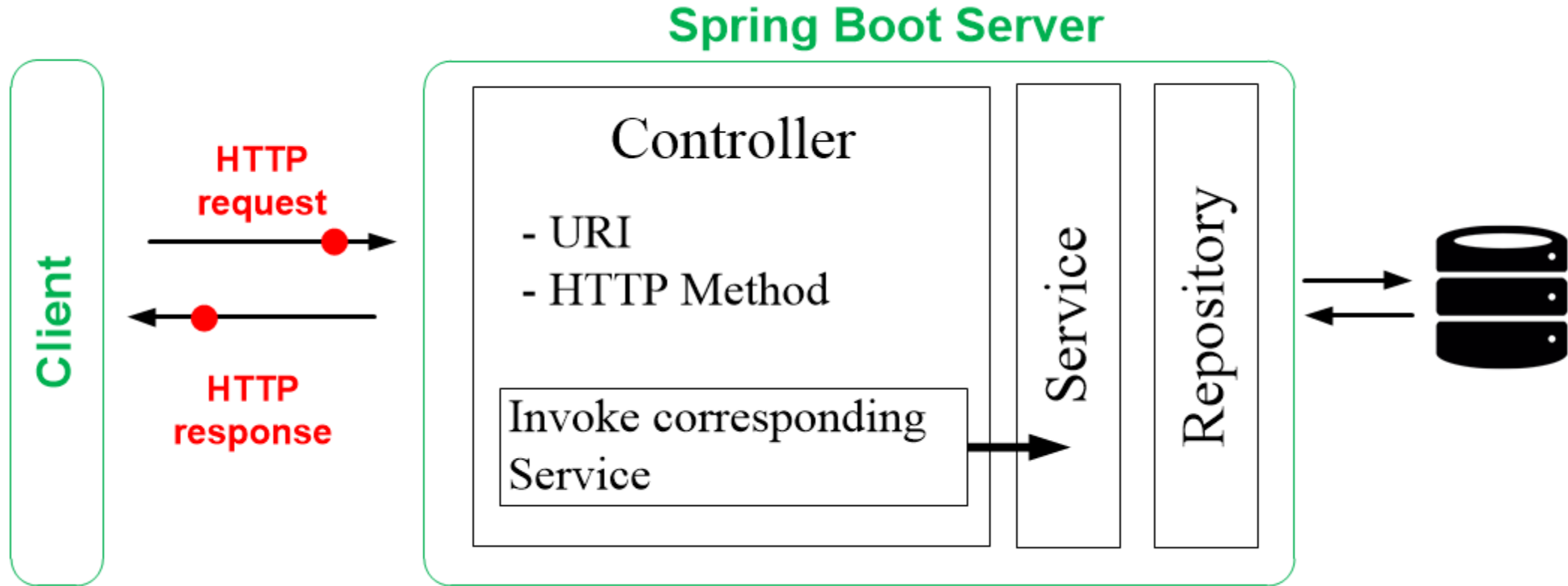
POST /products/**id**/orders

PUT /products/id/orders/**oid**

DELETE /products/id/orders/**oid**

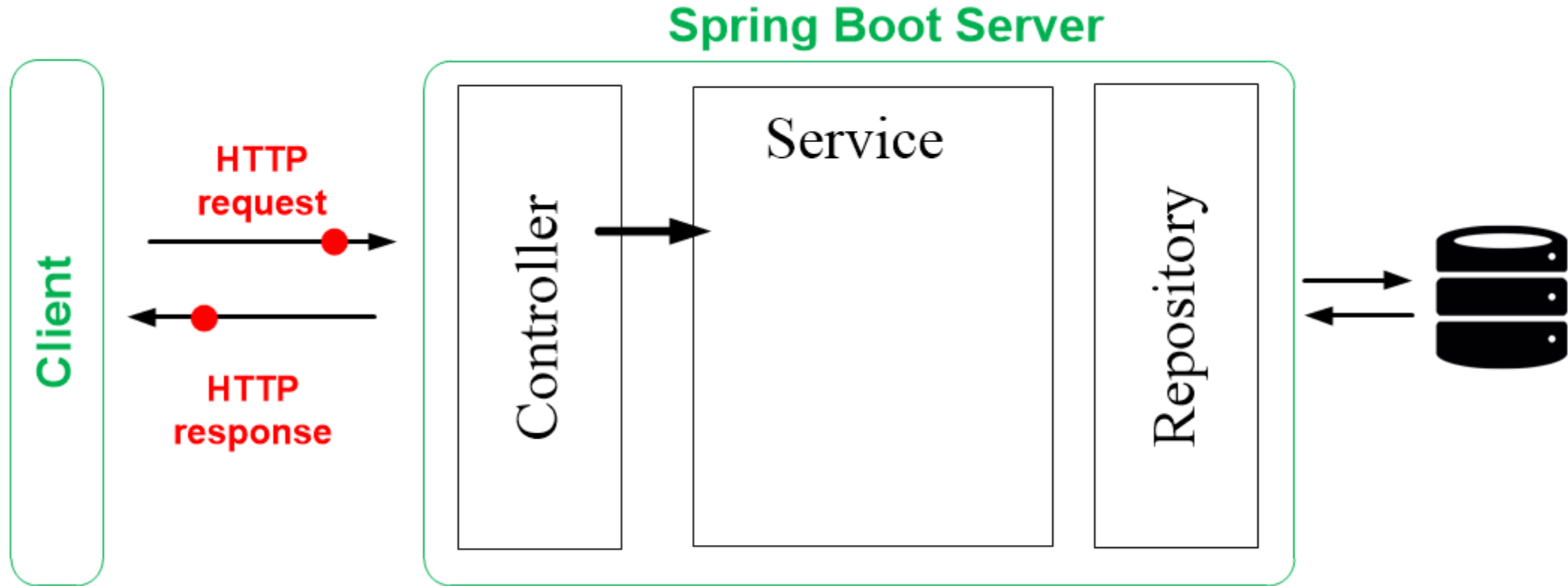
A **Controller** maps **clients requests** to corresponding **services/functionalities** (Java methods).

REST Architecture - Controller



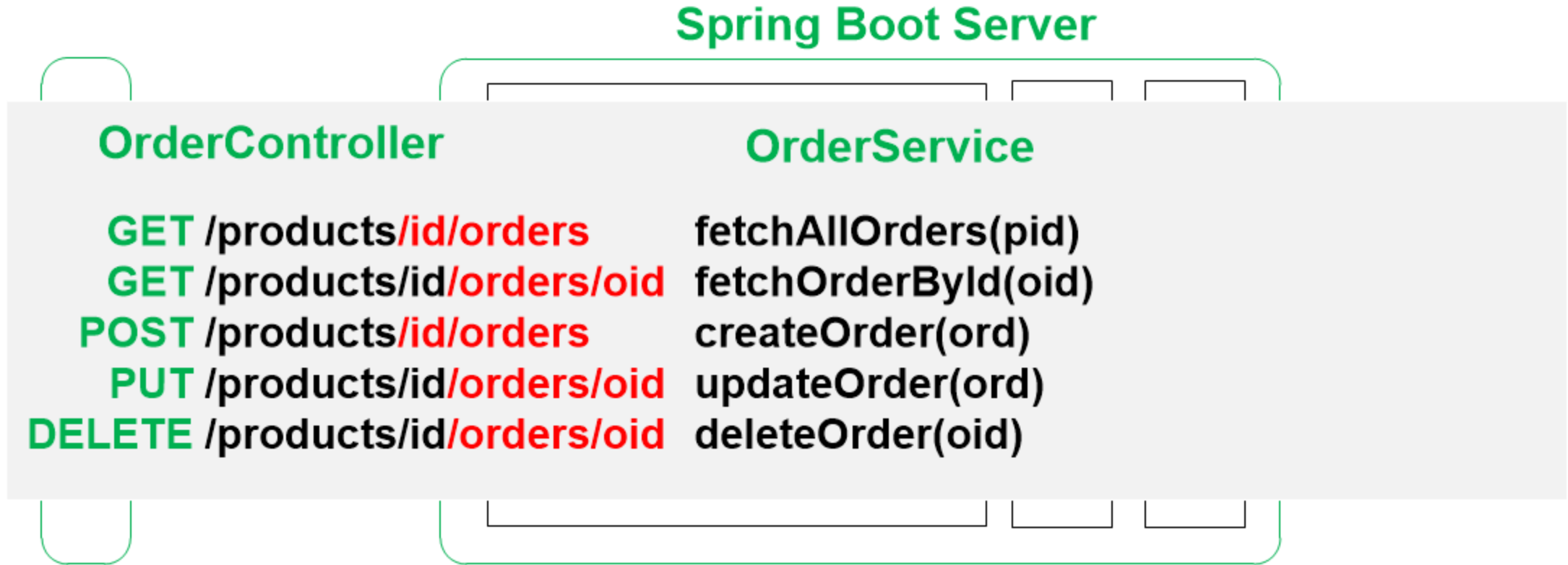
A **Controller** maps **clients requests** to corresponding **services/functionalities** (Java methods).

REST Architecture - Service



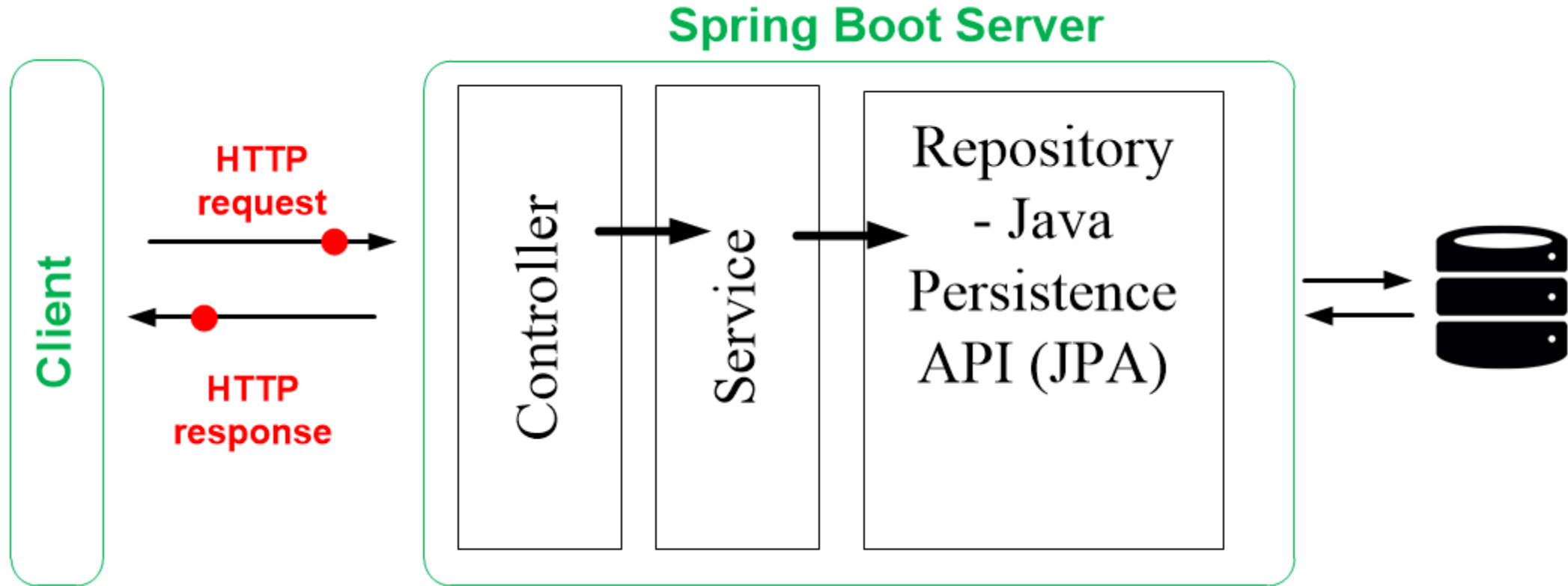
A Service (a method) implements the business logic underlining the service

REST Architecture - Service



A Service (a method) implements the business logic underlining the service

REST Architecture - Repository



A Repository is responsible for retrieving, storing, updating and deleting some set of data.

REST Architecture - Repository

Spring Boot Server

OrderController	OrderService	OrderRepository
GET /products/ id/orders	fetchAllOrders(pid)	ordRepo.findAll(pid)
GET /products/id/ orders/oid	fetchOrderById(oid)	ordRepo.findById(oid)
POST /products/ id/orders	createOrder(ord)	ordRepo.save(ord)
PUT /products/id/ orders/oid	updateOrder(ord)	ordRepo.save(ord)
DELETE /products/id/ orders/oid	deleteOrder(oid)	ordRepo.delete(oid)

JPA & **CRUD** Operations

The CSR pattern - practical example

ManyToOne

The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")
public class Order {

    @Id
    private String order_id;
    private String quantity;

    ..
    ..
}
```

The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")
public class Order {

    @Id
    private String order_id;
    private String quantity;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "id", nullable = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JsonIgnore
    private Product product;
    ..
    ..
}
```

The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")
public class Order {

    @Id
    private String order_id;
    private String quantity;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "id", nullable = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JsonIgnore
    private Product product;

    ..
    ..
}
```

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
/*indicates that the relation between a Product to
Order is many to one mapping. This means a product may
have a zero or many orders. However, an order can
belong to one product only, which exactly what we need.
*/
```

The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")
public class Order {

    @Id
    private String order_id;
    private String quantity;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id", nullable = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JsonIgnore
    private Product product;

    ..
    ..
}
```

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
```

```
/*indicates that the relation between a Product to
Order is many to one mapping. This means a product may
have a zero or many orders. However, an order can
belong to one product only, which exactly what we need.
*/
```

```
@JoinColumn(name = "id", nullable = false)
```

```
/* indicates Order will have id column which will
reference id of the product table. */
```

The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")
public class Order {

    @Id
    private String order_id;
    private String quantity;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id", nullable = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JsonIgnore
    private Product product;

    ..
    ..
}
```

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
```

```
/*indicates that the relation between a Product to
Order is many to one mapping. This means a product may
have a zero or many orders. However, an order can
belong to one product only, which exactly what we need.
*/
```

```
@JoinColumn(name = "id", nullable = false)
```

```
/* indicates Order will have id column which will
reference id of the product table. */
```

```
@OnDelete(action = OnDeleteAction.CASCADE)
```

```
/* indicates that if a parent product row is deleted in
the parent table, the corresponding order row will also
be deleted in the orders table. */
```

The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")
public class Order {

    @Id
    private String order_id;
    private String quantity;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id", nullable = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JsonIgnore
    private Product product;

    ..
    ..
}
```

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
/*indicates that the relation between a Product to
Order is many to one mapping. This means a product may
have a zero or many orders. However, an order can
belong to one product only, which exactly what we need.
*/
```

```
@JoinColumn(name = "id", nullable = false)
/* indicates Order will have id column which will
reference id of the product table. */
```

```
@OnDelete(action = OnDeleteAction.CASCADE)
/* indicates that if a parent product row is deleted in
the parent table, the corresponding order row will also
be deleted in the orders table. */
```

```
@JsonIgnore
/*is used to hide the field, when order is serialised
into json response body, which is sent to client. We
don't want the product details to be included in the
order response body.*/
```

The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")
public class OrderController {
    @Autowired
    private OrderService orderService;

    @GetMapping("/products/{productId}/orders")
    public List<Order> getAllOrders(@PathVariable String productId) {
        return orderService.getAllOrders(productId);
    }
    ..
    ..
    ..
}
```

The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")

package pu
    @RestController
    public class OrderController {

        @
        p
        p
        @Service
        public class OrderService {

            @Autowired
            @
            @
            private OrderRepository orderRepository;

            public List<Order> getAllOrders(String productId) {
                List<Order> orders = new ArrayList<>();
                orderRepository.findByProductId(productId).forEach(orders::add);
                return orders;
            }
        }
    }

    ..
    ..
}
```


The CSR pattern - practical example

```
@Entity
@Table(name = "orderstable")
package ...
import ...
import org.springframework.data.repository.CrudRepository;

public interface OrderRepository extends CrudRepository<Order, String>{
    public List<Order> findByProductId(String productId);
}

@Service
public class OrderService {
    // ...
}

@RestController
public class OrderController {
    // ...
}
```

Testing Spring Boot Applications With MockMvc and @WebMvcTest

What is MockMvc?

MockMvc is a **Spring Boot** test tool that allows **testing controllers** without needing to start an HTTP serve, i.e., **there is no real HTTP communication for such tests.**

With **MockMvc**, you will be working with a **mocked environment** provided by Spring and MockMvc acts as the entry point to this **mocked environment.**

What Can We Test with MockMvc?

MockMvc is a **Spring Boot** test tool that allows **testing controllers** without needing to start an HTTP server, i.e., **there is no real HTTP communication for such tests.**

With **MockMvc**, you will be working with a **mocked environment** provided by Spring and MockMvc acts as the entry point to this **mocked environment.**

We can test an API Endpoint with MockMvc

With **MockMvc**, we can perform several possible test cases:

- Test whether an API endpoint returns a specific **HTTP status code**,
- Test whether a REST API controller returns a **proper JSON response**,
- Test whether a REST API endpoint **is protected** with some **Authentication mechanism?**
- Test whether a user with a **specific Role** (e.g., Admin) can **access** a specific endpoint (e.g., update, delete)

MockMvc Setup

We only need to **annotate** our test with **@WebMvcTest**, which ensures to auto-configure **MockMvc** and also creates a **sliced Spring context** containing only **MVC-related beans**.

We can keep the sliced test context small, we can pass the class name of the controller we want to test: **@WebMvcTest(MyController.class)**. Otherwise, Spring will create a context including **all our controller endpoints**.

To verify that the controller handles HTTP requests, we call the **mockMvc.perform()** to initiate a **mock HTTP request**.

The mock requests are constructed using **builders** for different HTTP methods like **post()**, **get()**, **put()** and **delete()**. Taking it further, these builders take arguments (e.g., **contentType()** and **content()**).

MockMvc Setup

We only need to annotate our test with **@WebMvcTest**. This annotation not only ensures to auto-configure MockMvc but also creates a **sliced Spring context** containing only **MVC-related beans**.

```
@WebMvcTest(ProductController.class)
```

```
class ProductControllerTests {
```

```
    @MockBean
```

```
    private ProductController productController;
```

```
    @Autowired
```

```
    private MockMvc mockMvc;
```

```
    @Test
```

```
    public void getProductsApiTest() throws Exception  
    {
```

```
        mockMvc.perform(MockMvcRequestBuilders  
            .get("/api/products")  
            .accept(MediaType.APPLICATION_JSON))  
            .andExpect(status().isOk());
```

```
    }
```

```
    ..
```

```
}
```

We pass the class name of the controller we want to test

Spring Boot autoconfigures a MockMvc bean for us so that we can autowire it.

MockMvc Setup

We only need to annotate our test with **@WebMvcTest**. This annotation not only ensures to auto-configure MockMvc but also creates a **sliced Spring context** containing only **MVC-related beans**.

```
@WebMvcTest(ProductController.class)
```

```
class ProductControllerTests {
```

```
    @MockBean
```

```
    private ProductController productController;
```

```
    @Autowired
```

```
    private MockMvc mockMvc;
```

```
    @Test
```

```
    public void getProductsApiTest() throws Exception
```

```
    { We call the mockMvc.perform() to initiate a mock HTTP request.
```

```
        mockMvc.perform(MockMvcRequestBuilders
```

```
            .get("/api/products")
```

```
            .accept(MediaType.APPLICATION_JSON))
```

```
            .andExpect(status().isOk());
```

```
        // We can set some expectations through the andExpect()
```

```
    }
```

We pass the class name of the controller we want to test

Spring Boot autoconfigures a MockMvc bean for us so that we can autowire it.

We call the **mockMvc.perform()** to initiate a mock HTTP request.

Mock requests are constructed using **builders** for different HTTP methods **post()**, **get()**, **put()** and **delete()**.

MockMvc Setup

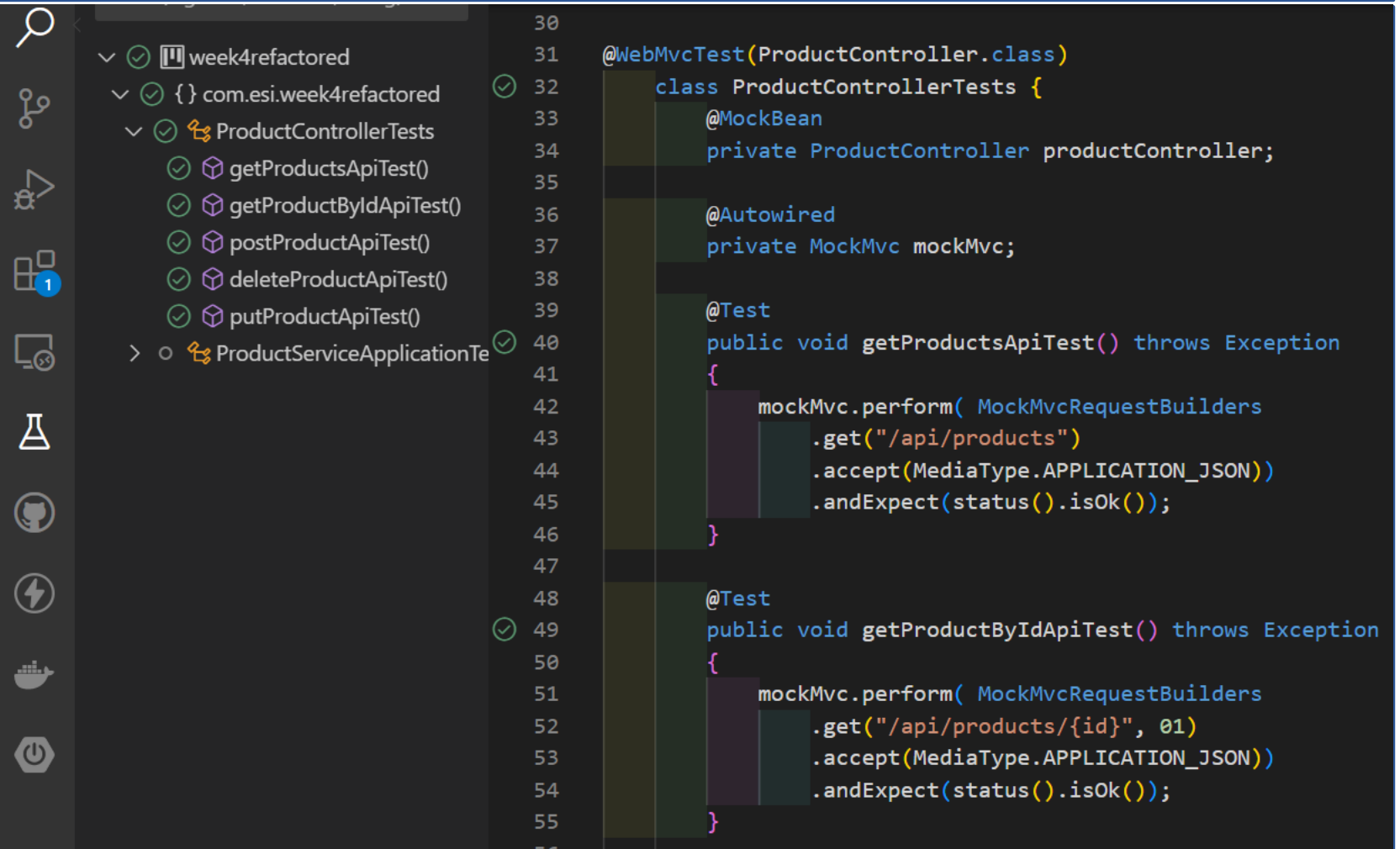
We only need to annotate our test with `@WebMvcTest`. This annotation not only ensures to auto-configure MockMvc but also creates a **sliced Spring context** containing only **MVC-related beans**.

```
@WebMvcTest(ProductController.class)
class ProductControllerTests {
    @MockBean
    private ProductController productController;

    @Autowired
    private MockMvc mockMvc;

    @Test
    public void getProductsApiTest() {
        We call the
        mockMvc.perform(
            .get("/api/products")
            .accept(MediaType.APPLICATION_JSON)
            .andExpect(status().isOk());
    }

    We can set some expectations
    the andExpect()
```



The screenshot shows an IDE with the following structure:

- Project: week4refactored
 - Package: com.esi.week4refactored
 - Class: ProductControllerTests
 - Method: getProductsApiTest()
 - Method: getProductByIdApiTest()
 - Method: postProductApiTest()
 - Method: deleteProductApiTest()
 - Method: putProductApiTest()
 - Class: ProductServiceApplicationTests

The code for `ProductControllerTests` is as follows:

```
30
31 @WebMvcTest(ProductController.class)
32 class ProductControllerTests {
33     @MockBean
34     private ProductController productController;
35
36     @Autowired
37     private MockMvc mockMvc;
38
39     @Test
40     public void getProductsApiTest() throws Exception
41     {
42         mockMvc.perform(MockMvcRequestBuilders
43             .get("/api/products")
44             .accept(MediaType.APPLICATION_JSON)
45             .andExpect(status().isOk());
46     }
47
48     @Test
49     public void getProductByIdApiTest() throws Exception
50     {
51         mockMvc.perform(MockMvcRequestBuilders
52             .get("/api/products/{id}", 01)
53             .accept(MediaType.APPLICATION_JSON)
54             .andExpect(status().isOk());
55     }
56 }
```


MockMvc Setup

We only need to annotate our test with **@WebMvcTest**. This annotation not only ensures to auto-configure MockMvc but also creates a **sliced Spring context** containing only **MVC-related beans**.

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {
    ...
    @Mock
    // You need to add this for the @ResponseStatus(HttpStatus.OK)
    private MockMvc mockMvc;

    @Autowired
    private ProductService productService;

    @GetMapping("/products")
    @ResponseStatus(HttpStatus.OK)
    public List<ProductDto> getAllProducts() {
        return productService.getAllProducts();
    }

    ...
}
```

ler we

a for

ders

We can set some expectations through the **andExpect()**

methods **post()**, **get()**, **put()** and **delete()**.

Best practices for good URI design

Uniform Resource Identifiers (URIs)

In **REST**, the primary data representation is called **resource**.

REST APIs use a **URI** to convey a **resource model** to the potential clients of the API.

When resources are **named well**, an API is intuitive and **easy to use**. If **done poorly**, that same API can be challenging to use and understand.

Singleton vs Collection Resources

A **resource** can be a singleton or a collection.

We can identify “products” collection resource using the URI
“/products”.

We can identify a **single** “product” **resource** using the URI
“/products/{productId}”.

Collection and Sub-collection Resources

A **resource** may contain **sub-collection resources**.

For example, **sub-collection resource** “orders” of a particular “products” can be identified using:

“/products/{productId}/orders”.

A **singleton resource** “reservation” inside the **sub-collection resource** “reservations” can be identified using:

“/products/{productId}/orders/{orderId}”.

Best practices for creating resource URIs

1. Use **nouns** to represent **resources**: a **resource** that is a **thing** (**noun**) not an **action** (**verb**) because nouns have **properties/attributes** that **verbs** do not have. `http://../api/products -catalog/..`
2. Use the “plural” name to denote the **collection resource**.
`http://../api/products -catalog/products`
3. Use forward slash (/) to indicate **hierarchical relationships** between resources.
`http://../products/{productId}/orders/{orderId}`
4. Never use **CRUD function names** in **URIs**: URIs should only be used to uniquely identify the resources and not any action upon them, which should be done using HTTP request methods.

Best practices for creating resource URIs

5. Do not use trailing forward slash (/)^{*} in URIs.

`http://../products-catalog/products/` /* adds no semantic value */

`http://../products-catalog/products` /* This is much better version */

6. Use hyphens (-) to improve the readability of URIs

`http://../productscatalog/products` /* not easy to read */

`http://../products-catalog/products` /* easier to read */

7. Do not use underscores (_) even it is possible to use an underscore in place of a hyphen as a separator, underscore (_) character can either get partially obscured or completely hidden in some browsers or screens.

^{*}A trailing slash is a forward slash ("/") placed at the end of a URL

OpenAPI-Swagger

Try it yourself ...



OpenAPI-Swagger



A back-end component offers APIs to be consumed by the front-end component. Thus, it is essential to have proper specifications for such APIs.

The API documentation should be informative, readable, and easy to follow. Moreover, reference documentation should simultaneously describe every change in the API. Accomplishing this manually is a tedious exercise, so automation of the process was inevitable.

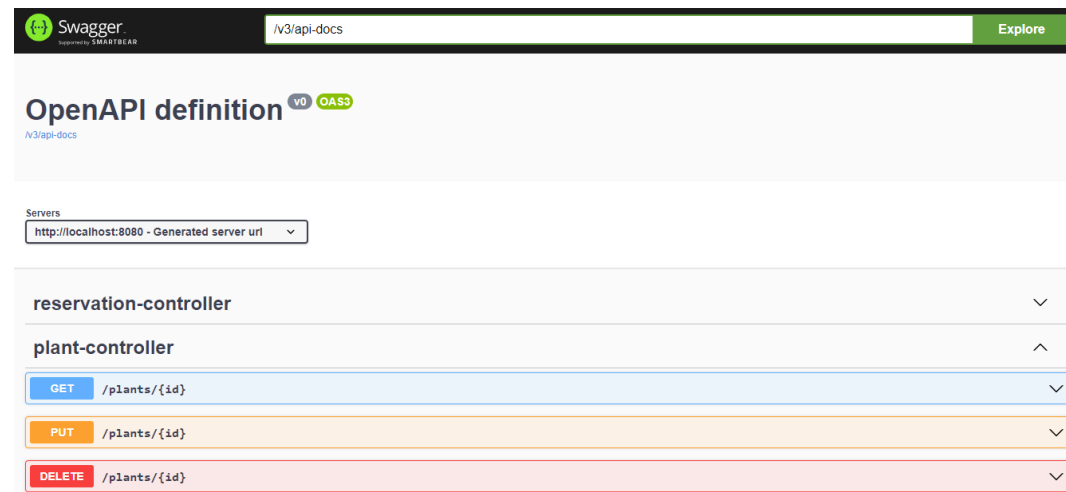
The OpenAPI Specification, previously known as the Swagger Specification, that is supported by the Springdoc project can solve this problem.

OpenAPI-Swagger

```
// Just add the Maven (springdoc-openapi) Dependency
// pom.xml
..
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.0.2</version>
    </dependency>
..
```

OpenAPI-Swagger

```
// Just add the Maven (springdoc-openapi) Dependency
// pom.xml
..
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.0.2</version>
</dependency>
..
```



OpenAPI-Swagger

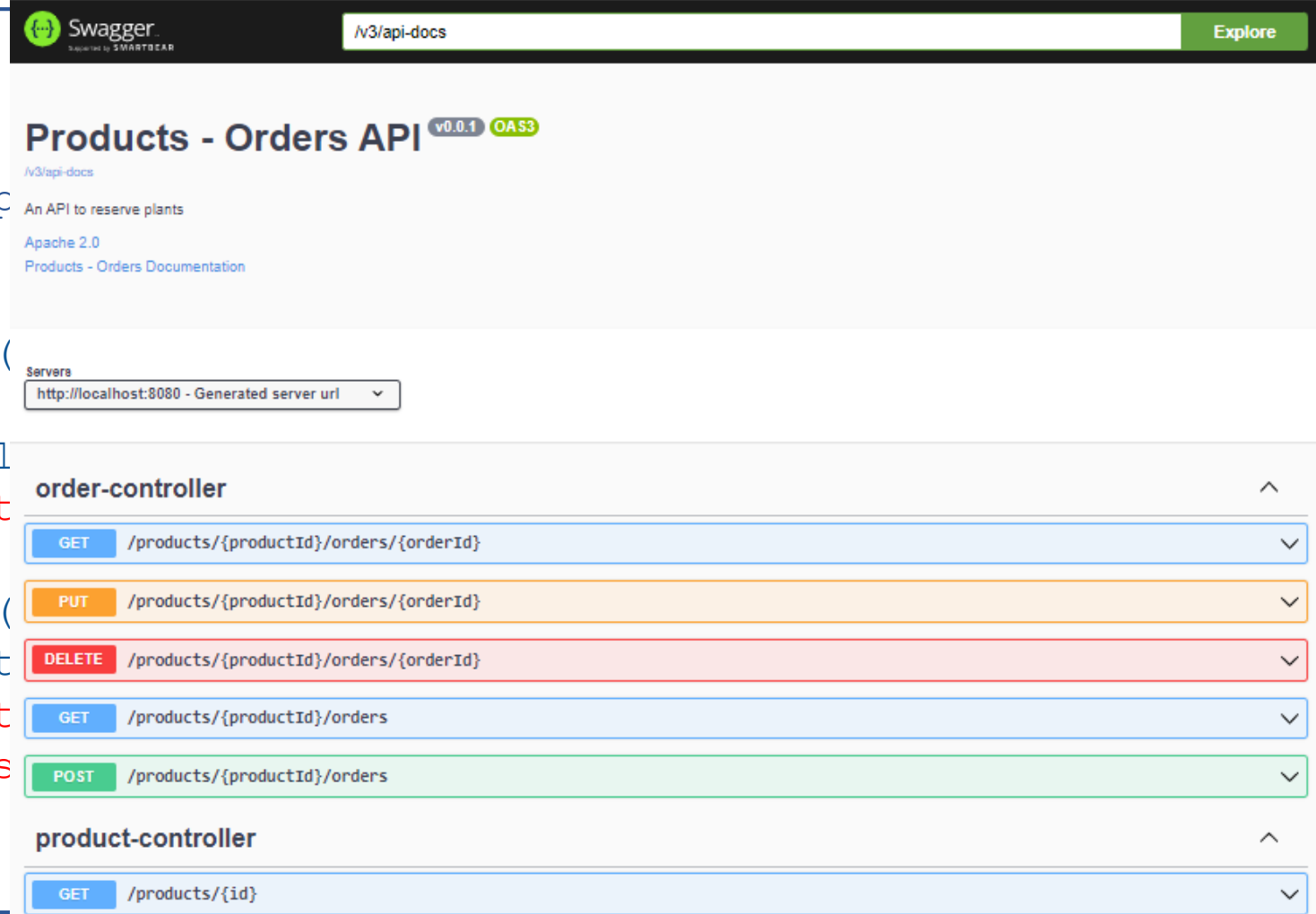
```
@SpringBootApplication
public class Week3Application {
    public static void main(String[] args) {
        SpringApplication.run(Week3Application.class, args);
    }
}

@Bean
public OpenAPI springShopOpenAPI() {
    return new OpenAPI()
        .info(new Info().title("Products - Orders API")
            .description("Products - Orders API")
            .version("v0.0.1")
            .license(new License().name("Apache 2.0").url("http://springdoc.org")))
        .externalDocs(new ExternalDocumentation()
            .description("Products - Orders Documentation")
            .url("http://localhost:8080/swagger-ui/4.15.5/index.html"));
}
}
```

OpenAPI-Swagger

```
@SpringBootApplication
public class Week4Application {
    public static void main(String[]
        SpringApplication.run(Week4Ap
    }
}

@Bean
public OpenAPI springShopOpenAPI()
    return new OpenAPI()
        .info(new Info().title
        .description("Product
        .version("v0.0.1")
        .license(new License(
        .externalDocs(new Ext
        .description("Product
        .url("http://localhos
    }
}
```

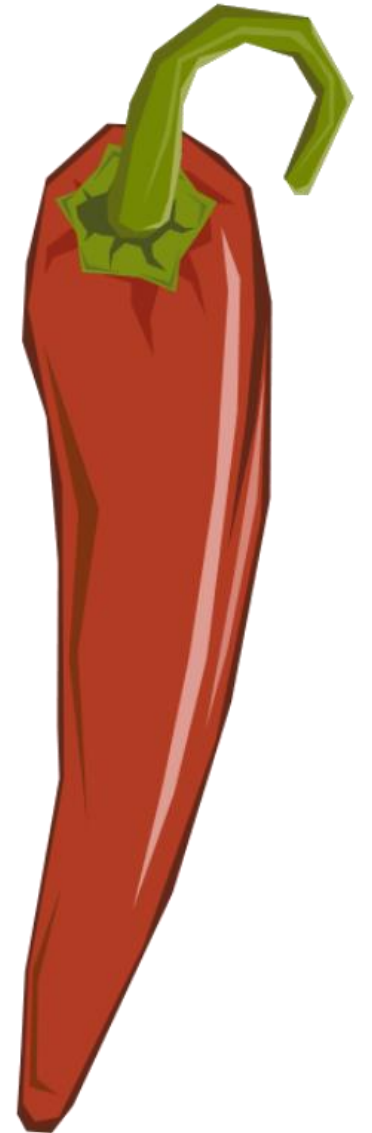
The image shows the Swagger UI interface for the 'Products - Orders API'. At the top, there's a header with the Swagger logo, the text 'Powered by SMARTCAR', and a search bar containing '/v3/api-docs' with an 'Explore' button. Below the header, the API title 'Products - Orders API' is displayed with version 'v0.0.1' and 'OAS3' tags. A description 'An API to reserve plants' and the license 'Apache 2.0' are shown, along with a link to 'Products - Orders Documentation'. A 'Servers' dropdown menu is set to 'http://localhost:8080 - Generated server url'. The main content area lists two controllers: 'order-controller' and 'product-controller'. The 'order-controller' has five endpoints: GET /products/{productId}/orders/{orderId}, PUT /products/{productId}/orders/{orderId}, DELETE /products/{productId}/orders/{orderId}, GET /products/{productId}/orders, and POST /products/{productId}/orders. The 'product-controller' has one endpoint: GET /products/{id}. Each endpoint is represented by a colored bar with a method icon (GET, PUT, DELETE, POST) and a dropdown arrow.

Project Lombok

Project Lombok

Project Lombok is a java library that automatically plugs into your editor and build tools, **spicing up your java**.

Never write another getter or equals method again, with one annotation your class has a fully featured builder, Automate your logging variables, and much more.



Project Lombok - Dependency

```
// Adding the Maven (project lombok) Dependency
// pom.xml
..
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.20</version>
    <scope>provided</scope>
</dependency>
..
```


Project Lombok - @Getter/@Setter

..

```
public class Product {  
    private String id;  
    private String name;  
    private String description;  
    private BigDecimal price;
```

..

```
public String getId() {  
    return this.id;  
}
```

```
public void setId(String id) {  
    this.id = id;  
}
```

..

Project Lombok - @Getter/@Setter

```
..  
@Getter @Setter  
public class Product {  
    private String id;  
    private String name;  
    private String description;  
    private BigDecimal price;  
  
..  
public String getId() {  
    return this.id;  
}  
  
public void setId(String id) {  
    this.id = id;  
}  
..
```

@Getter @Setter

You can annotate any field with **@Getter** and/or **@Setter**, to let lombok generate the default getter/setter automatically.

Project Lombok **@NoArgsC./@AllArgsC.**

```
..

public class Product {
    private String id;
    private String name;
    private String description;
    private BigDecimal price;

    public Product() {
    }

    public Product(String id, String name, String description, BigDecimal price) {
        this.id = id;
        this.name = name;
        ..
    }

    ..
}
```

Project Lombok **@NoArgsConstructor./@AllArgsConstructor.**

```
..  
@NoArgsConstructor @AllArgsConstructor
```

```
public class Product {  
    private String id;  
    private String name;  
    private String description;  
    private BigDecimal price;
```

```
public Product() {  
    }  
}
```

```
public Product(String id, String name, String description, BigDecimal price) {  
    this.id = id;  
    this.name = name;  
    ..  
}
```

```
..
```

@NoArgsConstructor will generate a constructor with no parameters.

@AllArgsConstructor generates a constructor with 1 parameter for each field in your class.

Project Lombok `@RequiredArgsConstructor`.

```
..
@RequiredArgsConstructor
public class Product {
    private String id;
    private String final name;
    private String description;
    @NonNull
    private BigDecimal price;
..
```

@RequiredArgsConstructor generates a constructor with 1 parameter for each field that requires special handling. All non-initialized **final** fields get a parameter, as well as any fields that are marked as **@NonNull** that aren't initialized where they are declared.

***A final field cannot have its value changed**

@NonNull can be used on a record component, or a parameter of a method or constructor, which will cause lombok to generate a null-check statement.

Project Lombok @RequiredArgsConstructor.

```
..
@RequiredArgsConstructor
public class Product {
    private String id;
    private String final name;
    private String description;
    @NonNull
    private BigDecimal price;
..
```

@RequiredArgsConstructor generates a constructor with 1 parameter for each field that requires special handling. All non-initialized **final** fields get a parameter, as well as any fields that are marked as **@NonNull** that aren't initialized where they are declared.

```
public class Product (String name, BigDecimal price){
    this.name = name;
    this.price = price;
}

..
Product product = new Product("milk", 1.8)
```

***A final field cannot have its value changed**

@NonNull can be used on a record component, or a parameter of a method or constructor, which will cause lombok to generate a null-check statement.

Project Lombok @Builder

```
..
@Builder
public class Product {
    private String id;
    private String name;
    private String description;
    private BigDecimal price;
..
```

The @Builder annotation produces complex builder APIs for your classes.

@Builder can be placed on a class, or on a constructor, or on a method.

Project Lombok @Builder

```
..
@Builder
public class Product {
    private String id;
    private String name;
    private String description;
    private BigDecimal price;
..
```

The **@Builder** annotation produces complex builder APIs for your classes.

@Builder can be placed on a class, or on a constructor, or on a method.

```
public void addProduct(ProductDto productDto) {

    Product product = Product.builder()
        .id(productDto.getId())
        .name(productDto.getName())
        .description(productDto.getDescription())
        .price(productDto.getPrice())
        .build();

    ..
}
```


Project Lombok @Data

```
..
@Data
public class Product {
    private String id;
    private String name;
    private String description;
    private BigDecimal price;
    ..
```

@Data is a convenient shortcut annotation that bundles the features of

- @ToString
- @EqualsAndHashCode
- @Getter
- @Setter
- @RequiredArgsConstructor

Project Lombok @Data

..

@ToString

```
public class Product {  
    private String id;  
    private String name;  
    private String description;  
    private BigDecimal price;  
    ..  
}
```

@ToString will cause lombok to generate an implementation of the `toString()` method, which allows getting the string representation of a Java object.

```
Output Product(id =1,name = 'John', ..)
```

Project Lombok @Data

```
..  
@ToString  
@ToString(includeFieldNames = false)  
public class Product {  
    private String id;  
    private String name;  
    private String description;  
    private BigDecimal price;  
..
```

@ToString will cause lombok to generate an implementation of the `toString()` method, which allows getting the string representation of a Java object.

```
Output Product(id =1,name = 'John', ..)
```

```
Output Product(1, 'John', ..)
```

Project Lombok @Data

```
..  
@EqualsAndHashCode  
public class Product {  
    private String id;  
    private String name;  
    private String description;  
    private BigDecimal price;  
..
```

@EqualsAndHashCode generate implementations of the `equals()` and `hashCode()`* methods.

```
new Product(1, "milk", ..).equals(new Product(1, "milk", ..))  
Output: True/false
```

```
..  
name.hashCode()  
..
```

*`hashCode()` returns an integer value, generated by a hashing algorithm.

Using best practices for our project

Better structure, DTOs, builders,

Using best practices for our project

products

- L `Product.java`
- L `ProductController.java`
- L `ProductService.java`
- L `ProductRepository.java`

Using best practices for our project

products

- └ Product.java
- └ ProductController.java
- └ ProductService.java
- └ ProductRepository.java

Products

- └ **model**
 - └ Product.java
- └ **dto**
 - └ ProductDto.java
- └ **controller**
 - └ ProductController.java
- └ **service**
 - └ ProductService.java
- └ **repository**
 - └ ProductRepository.java

Using best practices for our project

products

- L Product.java
- L ProductController.java
- L ProductService.java
- L ProductRepository.java

Products

- L **model**
 - L Product.java
- L **dto**
 - L ProductDto.java
- L **controller**
 - L ProductController.java
- L **service**
 - L ProductService.java
- L **repository**
 - L ProductRepository.java

Data Transfer Object (DTO) is an object that carries data between processes. Using DTO is a good and required practice.

The DTO is mainly used for reducing the number of expensive remote calls to RESTful APIs, hide implementation details of domain objects, control what we expose through endpoints to reduce security issues.

Using best practices for our project

products

- └ Product.java
- └ ProductController.java
- └ ProductService.java
- └ ProductRepository.java

Products

- └ **model**
 - └ Product.java
- └ **dto**
 - └ ProductDto.java
- └ **controller**
 - └ ProductController.java
- └ **service**
 - └ ProductService.java
- └ **repository**
 - └ ProductRepository.java

```
@RestController
@RequestMapping("/api")
public class ProductController {

    @Autowired
    private ProductService productService;

    @GetMapping("/products")
    public List<ProductDto> getAllProducts() {
        return productService.getAllProducts();
    }

    ...
}
```

Using best practices for our project

```
public List<Product> getAllProducts() {  
    List<Product> products = new ArrayList<>();  
    productRepository.findAll().forEach(products::add);  
    return products;  
}
```

Products

- └ **model**
 - └ Product.java
- └ **dto**
 - └ ProductDto.java
- └ **controller**
 - └ ProductController.java
- └ **service**
 - └ ProductService.java
- └ **repository**
 - └ ProductRepository.java

Using best practices for our project

```
@Builder
@Data
@AllArgsConstructor
@NoArgsConstructor
public class
ProductDto {

    ..

}
```

- L Product.java
- L **dto**
 - L ProductDto.java
- L **controller**
 - L ProductControll
- L **service**
 - L ProductService.
- L **repository**
 - L ProductRepository

```
public List<Product> getAllProducts() {

@Service

public class ProductService {

    public List<ProductDto> getAllProducts() {
        List<Product> products = new ArrayList<>();
        productRepository.findAll().forEach(products::add);
        return products.stream().map(this::mapToProductDto).toList();

    }

    private ProductDto mapToProductDto(Product product) {
        return ProductDto.builder()
            .id(product.getId())
            .name(product.getName())
            .description(product.getDescription())
            .price(product.getPrice())
            .build();

    }
```

Using best practices for our project

```
@Builder
@Data
@AllArgsConstructor
@NoArgsConstructor
public class
ProductDto {

    ..

}
```

- L Product.java
- L **dto**
 - L ProductDto.java
- L **controller**
 - L ProductControll
- L **service**
 - L ProductService.
- L **repository**
 - L ProductReposito

```
public List<Product> getAllProducts() {

@Service
@Slf4j
public class ProductService {

    public List<ProductDto> getAllProducts() {
        List<Product> products = new ArrayList<>();
        productRepository.findAll().forEach(products::add);
        return products.stream().map(this::mapToProductDto).toList();
        log.info("A GET all request has arrived");

#application.properties
...
logging.level.com=INFO

    public ProductDto mapToProduct(Product product) {

        .id(product.getId())
        .name(product.getName())
        .description(product.getDescription())
        .price(product.getPrice())
        .build();

    }

}
```




UNIVERSITY OF TARTU

**Thank You
for your attention**

Mohamad Gharib
mohamad.gharib@ut.ee



unitartu



tartuuniversity

